

SentinelCut — API Contract (v1, MVP)

This is the fixed contract all four team members build against. Backend implements it exactly as written here; frontend, AI, and audio/video pieces integrate against it without needing to ask each other questions. If something needs to change, update this file first, then code — not the other way around.

Base URL (local dev): `http://localhost:8000/api/v1`

Auth: all endpoints except `/auth/*` require `Authorization: Bearer <jwt>` header.

1. Auth

`POST /auth/signup`

Request

```
json
{ "email": "user@example.com", "password": "min8chars" }
```

Response 201

```
json
{ "user_id": "uuid", "email": "user@example.com", "access_token": "jwt..." }
```

`POST /auth/login`

Request

```
json
{ "email": "user@example.com", "password": "min8chars" }
```

Response 200

```
json
{ "user_id": "uuid", "email": "user@example.com", "access_token": "jwt..." }
```

Response 401 — invalid credentials

2. Upload

`POST /upload`

`multipart/form-data`

- `file`: the audio/video file (mp4, mp3, wav for MVP)
- `custom_words` (optional): JSON string array, e.g. `["badword1", "badword2"]`

Response 202 — file accepted, job queued

```
json
{
  "job_id": "uuid",
  "file_id": "uuid",
  "filename": "clip.mp4",
  "status": "queued"
}
```

Response 400 — bad format / file too large

```
json
{ "error": "unsupported_format", "message": "Only mp4, mp3, wav are supported in
```

3. Job status

`GET /jobs/{job_id}`

Response 200

```
json
{
  "job_id": "uuid",
  "status": "queued | processing | done | failed",
  "progress_pct": 0,
  "stage": "transcribing | detecting | rendering | null",
  "error": null
}
```

`status` values and meaning:

- `queued` — accepted, not started
- `processing` — worker picked it up, see `stage`
- `done` — detections + outputs ready
- `failed` — see `error`

Frontend polls this every 2–3s (per your NFR: status updates within 5s).

4. Detections

GET /jobs/{job_id}/detections

Only valid once `status = done` or `processing` (partial results allowed once transcription stage completes).

Response 200

json

```
{
  "job_id": "uuid",
  "transcript_language_hint": "hinglish",
  "detections": [
    {
      "id": "det_001",
      "word": "fuuuuck",
      "normalized": "fuck",
      "language": "english",
      "severity": "high",
      "start": 12.34,
      "end": 12.81,
      "source": "dictionary",
      "confidence": 0.95,
      "replacement": "beep",
      "status": "active"
    }
  ],
  "full_transcript": [
    { "word": "this", "start": 12.01, "end": 12.20 },
    { "word": "movie", "start": 12.21, "end": 12.33 }
  ]
}
```

See `detection-schema.json` for the authoritative schema of a single detection object.

`status` on a detection: `active` (will be censored) | `removed` (user deleted it) | `added` (user manually added it).

PATCH /jobs/{job_id}/detections/{detection_id}

User edits a detection in the timeline editor. **Request** (any subset of fields)

json

```
{ "status": "removed" }
```

or

```
json  
  
{ "replacement": "mute" }
```

Response 200 — updated detection object.

POST /jobs/{job_id}/detections

User manually adds a detection the AI missed. **Request**

```
json  
  
{ "word": "badword", "start": 45.0, "end": 45.5, "replacement": "beep" }
```

Response 201 — created detection object, `{source: "manual", status: "added"}`.

5. Regenerate / export

POST /jobs/{job_id}/regenerate

Re-renders the output using current detection edits. Triggers a new render job (does not redo transcription/detection). **Response 202**

```
json  
  
{ "job_id": "uuid", "status": "processing", "stage": "rendering" }
```

GET /jobs/{job_id}/export?type=video|audio|transcript|json

Returns a signed download URL for the requested artifact. **Response 200**

```
json  
  
{ "type": "video", "download_url": "https://storage.../signed-url", "expires_at":
```



6. Dashboard

GET /files

List current user's uploads (recent files / history — MVP version, no pagination needed yet). **Response 200**

```
json
```

```
{
  "files": [
    { "job_id": "uuid", "filename": "clip.mp4", "status": "done", "created_at": "2020-01-01T00:00:00Z" }
  ]
}
```

7. Error format (all endpoints)

json

```
{ "error": "error_code", "message": "human readable message" }
```

Common `error` codes: `unsupported_format`, `file_too_large`, `unauthorized`, `not_found`, `processing_failed`.

Notes for each owner

- **Person 1 (AI):** your script's final output must map directly into the `detections` array shape below — that's the handoff point to Person 2.
- **Person 2 (backend):** implement all endpoints above with mock/hardcoded data first (Step 4), then wire to Person 1's pipeline and Person 3's renderer.
- **Person 3 (audio/video):** your render function's input is the `detections` array (only `status: active` ones) + original file; output feeds the `/export` endpoint.
- **Person 4 (frontend):** build against this contract directly; use the example JSON blocks above as your mock data while backend is still wiring up.